Software Safety and Halstead's Software Science (Lecture 16)

Dr. H S Behera

Organization of this Lecture:

- Software safety:
  - General concepts
  - Fault avoidance
  - Fault detection
  - Fault-tolerance
- Halstead's software science
- Summary

Fault-tolerance concepts

- Fail safe:
  - design the system so that when it sustains a specified fault
    - it fails in a safe mode
  - railway signalling systems are designed to be fail safe
    - so that all trains stop.

Fault-tolerance concepts

- Fail stop:
  - when the system sustains specified faults:
    - provides a subset of its required behavior

Software safety

- Higher reliability through 3 complementary strategies:
  - fault avoidance:
    - design time
  - fault tolerance:
    - execution time
  - fault detection:
    - implementation and testing time

Fault detection

- Faults are detected before software put to operation.
  - Verification
  - validation
    - static
    - dynamic

Fault avoidance

- Fault avoidance relies on several schemes.
  - appropriately tune
    - design and implementation process.

Fault avoidance

- Precise (and preferably formal) specification
- Adoption of quality principles
- Adoption of a design strategy based on information hiding
- Use of a strongly typed programming language
- Restriction on error-prone programming constructs such as pointers.

Fault tolerance

- Assumes some residual faults remain.
  - Facilities are provided in software to allow operation to proceed even when faults surface.

Strongly typed languages

- Strongly typed languages:
  - can detect many of the faults at compile time.
- If low level programming with limited type checking is used.
  - Fault free software production is virtually impossible.

Fault-tolerance

- Consists of several aspects:
  - fault detection
  - fault diagnosis
  - damage assessment and containment
  - fault recovery
  - fault repair

Fault detection

- The system must detect:
  - a particular state combination has resulted or will result in system failure.
  - The process of determining that a fault has occurred.

Fault diagnosis

- The process of determining what caused the fault:
  - i.e. exactly which subsystem or component is faulty.

Damage assessment and containment

- Detect parts of system:
  - affected by failure.
- Prevent propagation of faults.

Fault recovery

- The system must restore its state to a known safe state.
- Two options available:
  - correct the damaged state (forward error recovery)
  - restore the system to a known safe state (backward error recovery)

Fault recovery

- Forward error recovery is more complex:
  - involves diagnosing faults:
    - knowing in what state the system should have been:
      - had the system not failed.

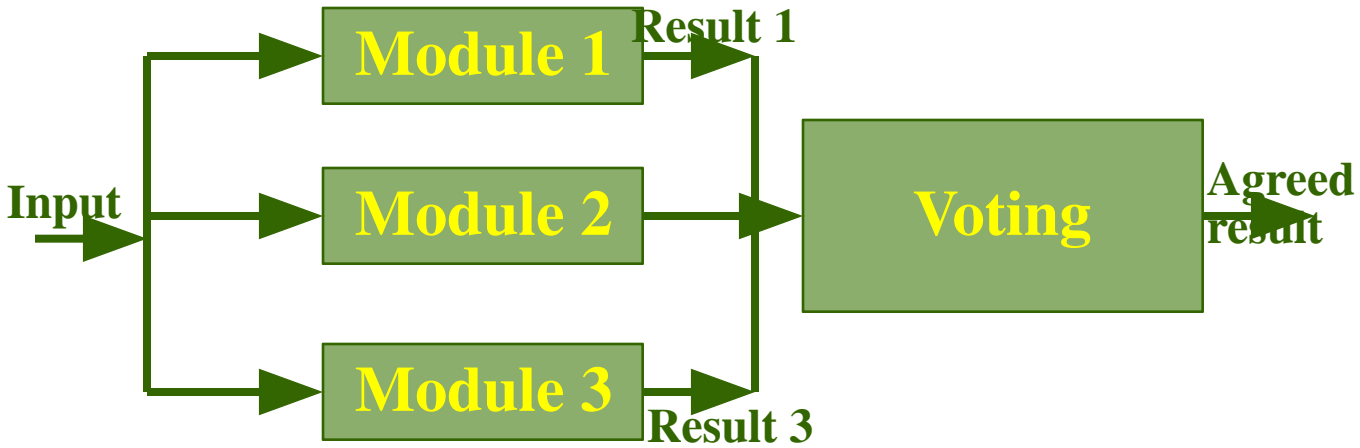
Fault repair

- Involves modifying the system:
  - In many cases, software failures are transient:
    - occur due to peculiar combination of system inputs
    - no repair is necessary as normal processing can continue immediately after fault recovery.

Hardware fault tolerance

- Most commonly used hardware fault-tolerance technique:
  - triple modular redundancy (TMR)

Triple modular redundancy (TMR)



Triple modular redundancy

- Hardware module is replicated three times:
  - output from each unit is compared
  - voting is used to determine the correct result.

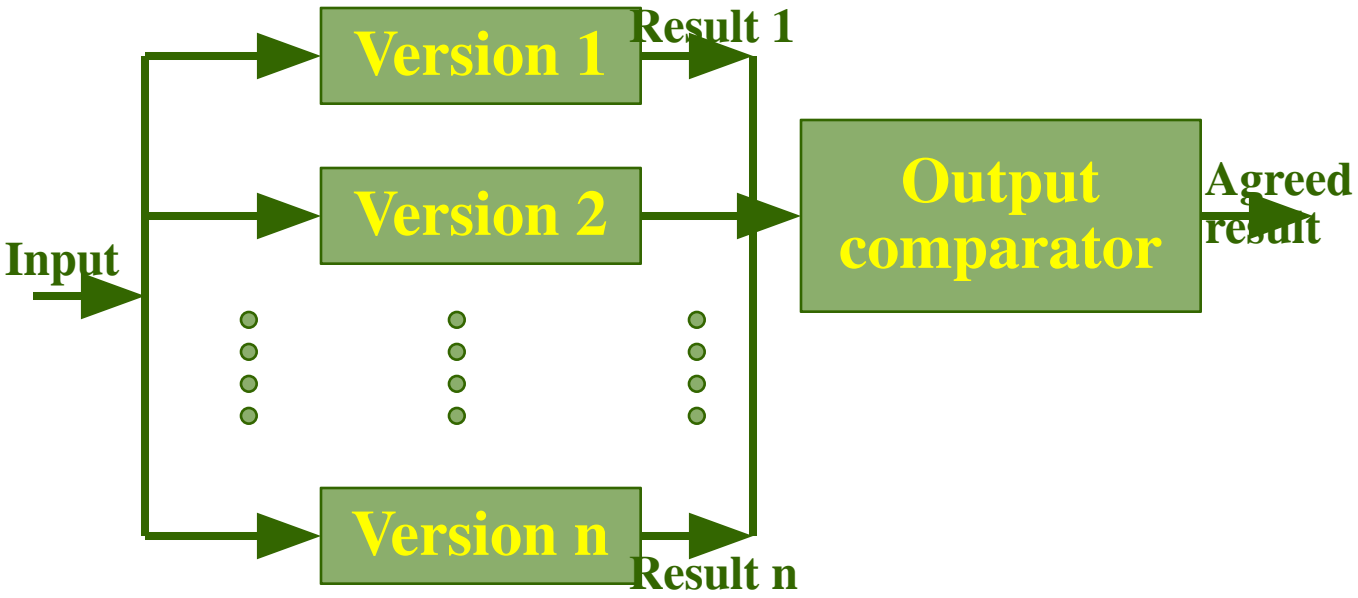
Software fault-tolerance

- N-version programming
- Recovery blocks

N-version programming

- Different versions of the software
  - implemented by different teams
    - executed in parallel
  - outputs compared using a voting system:
    - inconsistent outputs are rejected.

N-version programming



N-version programming

- At least three versions of the software should be available
- The basic assumption:
  - versions developed by different engineers would not have similar errors.

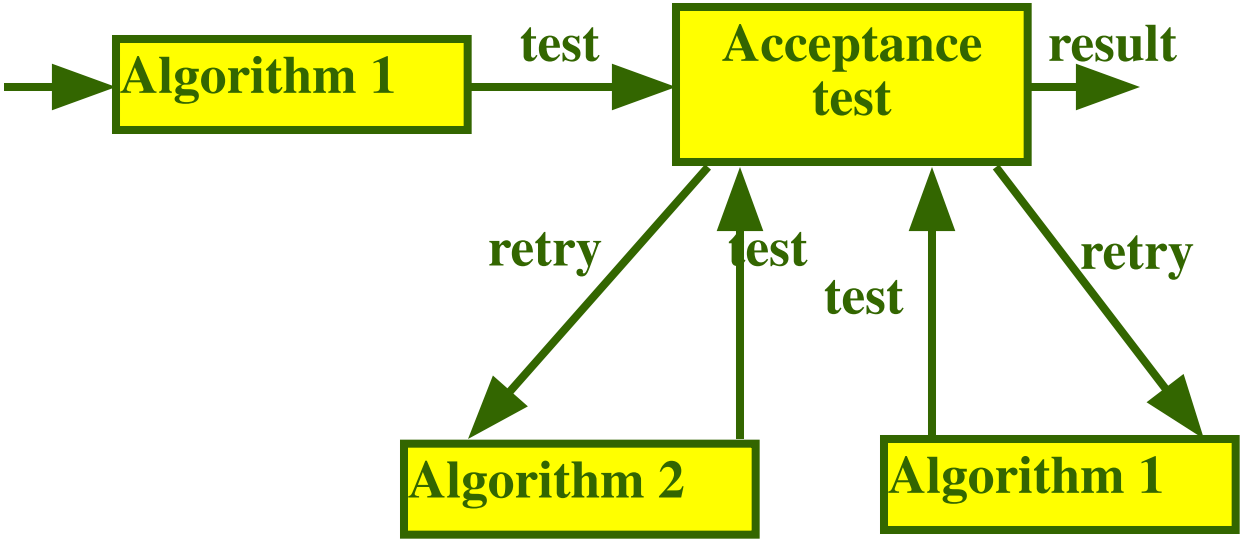
Recovery blocks

- A fine grain approach
- Each program component includes
  - an acceptance test:
    - checks if it executed successfully.
    - Acceptance tests cannot determine what has gone wrong
  - The program components are called
    - try blocks or recovery blocks.

Recovery blocks

- Include alternative code:
  - system can backup and execute alternative code
  - recovery blocks can be cascaded
    - multiple alternatives can be tried when an alternate result also fails acceptance test.

Recovery blocks



Comparison of n-version programming and recovery block

- Unlike n-version programming
  - alternative code is different rather than independent
  - Also alternative is executed in sequence rather than parallel
- Both suffer from common mode failure.

Exception handling

- An exception is an error or an unexpected event
- When an exception has not been anticipated:
  - control is transferred to the system exception handling mechanism

Exception handling

- Many programming languages do not have facility:
  - to detect and handle exceptions.
- Languages which support exception handling:
  - Ada, C++, Java

Fault detection and damage assessment

- The techniques to be used are dependent on the application and system state:
  - use of checksums in data exchange and check digits in numeric data
  - use of redundant links in data structures which contain pointers
  - use of watchdog timers in concurrent systems.

Checksums

- Checksum value computed:
  - by applying some mathematical function to the data.
  - The function should give unique value for the data.

Checksums

- Sender computes the checksum:
  - sends the data with the checksum value
  - receiver computes the checksum again
  - if the two checksum values differ,
    - the receiver knows that some data have been corrupted.

Watchdog timers

- Used when a function must complete within a specific time period.
- Watchdog timer is a timer:
  - must be reset by the function after it completes execution.
  - may be interrogated by a controller at regular intervals
  - if for some reason,
    - the function does not terminate,
    - the watchdog timer is not reset.

Fault recovery

- Modify system configuration
  - so that system can continue operation
    - perhaps in some degraded form
    - forward recovery
    - backward recovery

Forward recovery

- Correct damaged system state
  - use redundant information with
  - Data corruption: Use coding technique which add redundant information
  - Corruption of linked structures: include redundant pointers, e.g both forward and backward pointers.

Backward error recovery

- Restores system state to a known safe state.
  - Simpler technique than forward error recovery.
  - most database systems include backward error recovery.

Example backward recovery

- Computations for a specific user request is known as a transaction.
  - Changes made during a transaction are not immediately incorporated
  - changes made only after all computations involved in the transaction finish successfully.

Checkpointing

- Transactions allow error recovery
  - because they do not commit changes to the database until they are completed.
  - However, they do not allow recovery from system states that are valid but incorrect.
  - Checkpointing can be used.

Checkpointing

- The system state is duplicated periodically.
- When a problem is discovered
  - correct state can be recovered.

Safety life cycle

- Hazard:
  - A condition with the potential for causing or contributing to a mishap.
  - e.g, A failure of the sensor which detects an obstacle in front of a machine

Hazard Analysis

- Analyze the system and its environment
  - detect causes of hazards.
- Hazard analysis is difficult
  - because many hazards are extremely rare.

Fault-tree analysis

- For each identified hazard:
  - a detailed analysis is carried out to discover the conditions which might cause the hazard.
  - Fault-tree analysis involves identifying the undesired event
    - working backwards from the event to discover the possible causes.

Fault-tree analysis

- The hazard is at the root of the tree:
  - leaves represent potential causes of hazards.

Halstead's Software Science

- An analytical technique to measure:
  - size, development effort, and development time.

Halstead's Software Science

- Halstead used primitive program parameters:
  - developed expressions for:
    - over all program length,
    - potential minimum volume,
    - actual volume,
    - language level,
    - effort,
    - development time.

Halstead's Software Science (CONT.)

- For some given program, let:

  η_1 be the number of unique operators used in the program,

  η_2 be the number of unique operands used in the program,

- N1 be the total number of operators used in the program,
- N2 be the total number of operands used in the program.

Halstead's Software Science (CONT.)

- The terms operators and operands have intuitive meanings,
  - a precise definition of these terms is needed to avoid ambiguities.
  - Unfortunately there is no general agreement among researchers
    - on definition of operators and operands:

Operators

- Some general guidelines can be provided:
  - All assignment, arithmetic, and logical operators are operators.
  - A pair of parentheses,
    - as well as a block begin --- block end pair, are considered as single operators.
  - A label is considered to be an operator,
    - if it is used as the target of a GOTO statement.

Operators

- An `if ... then ... else ... endif` and a `while ... do` construct are single operators.
- A sequence (statement termination) operator ';' is a single operator.
- function call
  - Function name is an operator,
  - I/O parameters are considered as operands.

Halstead's Software Science (CONT.)

- The set of operators and operands for the ANSI C language: `() [] . , -> * + - ~ ! ++ -- * / % + - << >> < > <= >= != == & ^ | && || = *= /= %= += -= <<= >>= &= ^= |= : ? { ; CASE DEFAULT IF ELSE SWITCH WHILE DO FOR GOTO CONTINUE BREAK RETURN` and a function name in a

Halstead's Software Science (CONT.)

- Operands are those variables and constants
  - which are being used with operators in expressions.
- Note that variable names appearing in declarations
  - are not considered as operands.

Examples:

- In the expression `a = &b;`
  - {a, b} are operators and
  - { =, &} are operands.

Examples:

- The function name in a function definition
  - not counted as an operator.
  - int func (int a, int b)
    { . . .
    }
    - the operators are: {}, ( )
    - We do not consider func, a, and b as operands.

Examples (CONT.):

- In the function call statement:
  func (a, b);
  - {func ( ) ;} are considered as a operator
  - variables a, b are treated as operands.

Length and Vocabulary

- Length of a program quantifies
  - total usage of all operators and operands in the program:
  - Thus, length N=N1+N2.
- Program vocabulary:
  - number of unique operators and operands used in the program.
  - program vocabulary $\eta = \eta_1 + \eta_2$.

Program Volume:

- The length of a program:
  - total number of operators and operands used in the code
  - depends on the choice of the operators and operands,
    - i.e. for the same program, the length depends on the style of programming.

Program Volume:

- We can have highly different measures of length
  - for essentially the same problem.
- To avoid this kind of problem,
  - the notion of program volume V is introduced:
  - $V = N \log_2 \eta$

Potential Minimum Volume:

- Intuitively, program volume V denotes
  - minimum number of bits needed to encode the program.
- To represent η different identifiers,
  - we need at least log2 η bits (η is the program vocabulary)

Potential Minimum Volume:

- The potential minimum volume V*:
  - volume of the most succinct program in which the program can be coded.

Potential Minimum Volume:

- Minimum volume is obtained :
  - when the program can be expressed using a single source code instruction:
    - say a function call like foo().

Potential Minimum Volume (CONT.):

- Lower bound on volume:
  - a program would have at least two operators
  - and no less than the requisite number of operands (i.e. input/ output data items).

Potential Minimum Volume:

- If an algorithm operates on input/ output data d1, d2,... dn,
  - the most succinct program is f(d1,d2, . ..,dn);
  - for which $\eta_1 = 2$, $\eta_2 = n$
- Therefore, V*=(2+ 2) log2(2+ 2)

Potential Minimum Volume:

- The program level L is given by L=V*/V.
- L is a measure of the level of abstraction:
  - languages can be ranked into levels that appear intuitively correct.

Effort and Time:

- Effort E=V/L, where
  - E is the number of mental discriminations required to write the program
  - also the effort required to read and understand the program.

Effort and Time:

- Thus, programming effort E = V2/ V*
  - since L= V*/V varies as the square of the volume.
- Experience shows
  - E is well correlated to the effort needed for maintenance.

Effort and Time:

- The programmer's time T=E/S,
  - where S is the speed of mental discriminations developed from psychological results due to Stroud,
  - the recommended value for software is 18.

Length Estimation:

- Halstead assumed that it is quite unlikely that a program has several identical parts ---
  - or substrings of length greater than (η being the program vocabulary).
-

Length Estimation:

- In fact, once a piece of code occurs identically in several places,
  - it is usually made into a procedure or a function.
- Thus, we can safely assume:
  - any program of length N consists of N/η unique strings of length .

Length Estimation (CONT.):

- It is a standard combinatorial result that for any given alphabet of size K,
  - there are exactly Kr different strings of length r.
- Thus, $N/\eta < \eta$
- Or, $N < \eta^{\eta+1}$

Length Estimation:

- Since operators and operands usually alternate in a program,
  - we can further refine the upper bound into

    $N < (\eta_1)^{\eta_1} (\eta_2)^{\eta_2}$.

Length Estimation (CONT.):

- Also, N must include not only the ordered set of N elements,
  - but it must also include all possible subsets of that ordered set,
  - i.e. the power set of N strings.
  - Therefore, $2^N = (\eta_1)^{\eta_1} (\eta_2)^{\eta_2}$.

Length Estimation:

- $N = \log_2((\eta_1)^{\eta_1} (\eta_2)^{\eta_2})$ (approximately)
- Or, $N = \log_2 (\eta_1)^{\eta_1} + \log_2 (\eta_2)^{\eta_2}$

$$= \eta_1 \log_2 \eta_1 + \eta_2 \log_2 \eta_2$$

- Experimental analysis of large number of programs suggests:
  - computed and actual lengths match very closely.

Example:

```
main()
{
    int a,b,c,avg;
    scanf("%d %d %d",&a,&b,
&c);
    avg=(a+b+c)/3;
    printf("avg= %d",avg);
}
```

Example:

- The unique operators are: main, (), \{\}, int, scanf, \&, ",", ";", =, +, /, printf
- The unique operands are: a,b,c, \&a,\&b,\&c,a+b+c,avg,3,"\%d \%d \%d", "avg=\%d”

Example (CONT.):

- Therefore 1=12, 2=11
- Estimated Length=(12*log12+11*log11)
  =(12*3.58 + 11*3.45)
  =(43+38)=81
  Volume=Length*log(23)=81*4.52=3
  66

Summary

- High reliability achieved through 3 complementary strategies:
  - fault avoidance
  - fault tolerance
  - fault detection
- Fault tolerance:
  - n-version programming
  - recovery blocks

Summary

- Halstead's software science
  - analytical method.
  - Lets us determine:
    - length
    - volume
    - effort
    - time